CNN/Siamese Embedding Endpoints

Integration guide for the Flutter team — a second, independent matching path (single-finger and slap) alongside the existing minutiae endpoints

1. What this is

Six new endpoints (three single-finger, three slap) on the same cloud service you already talk to, offering an alternative to minutiae-based matching: a CNN (pretrained ResNet18) converts each fingerprint image into a 512-number vector ("embedding"), and two images are compared by how similar their vectors are (cosine similarity), instead of comparing extracted ridge-point locations. The single-finger version is validated at 92.5% matching accuracy on a 336-person test set (vs. ~52% for the existing minutiae matcher) — see the attached *Siamese_Embedding_Phase_A_Report.pdf* for the full methodology and numbers. The slap version uses the exact same per-finger matcher and aggregation shape as the existing minutiae slap endpoints, but has **not** been separately measured for slap-specific accuracy/FAR/FRR — treat it as functionally verified, not yet accuracy-validated.

This runs **alongside** the existing endpoints, not instead of them. Nothing about /v2/enroll, /v2/authenticate, /v2/verify, /v2/enroll_slap, /v2/authenticate_slap, or /v2/verify_slap changes — they still work exactly as before, on their own separate data tables.

2. Base URL

`https://34-100-150-103.sslip.io`

Same host as the existing /v2/* minutiae endpoints.

3. Single-finger endpoints

POST /v2/enroll_embedding

Multipart form fields:

Field	Required	Notes
image	Yes	The preprocessed fingerprint image (same as you'd send to /v2/enroll)
name	Yes	Person's name
uid	Yes	Unique person identifier
batch	Yes	Batch/group identifier — authenticate matches within one batch
finger_position	No	Integer 0-10, default 0

Response (success):

```
{"success": true, "embedding_dim": 512}
```

Response (error, e.g. missing field or bad image):

```
{"success": false, "error": "..."}
```

POST /v2/authenticate_embedding — 1:N (identify)

Multipart fields: **batch**, **image**. Compares the image against every person enrolled in that batch, returns the best match.

Response (match found):

```
{"success": true, "name": "...", "uid": "...", "confidence": 0.9954, "threshold": 0.94}
```

Response (no match / below threshold):

```
{"success": false, "message": "No match found", "confidence": 0.6123, "threshold": 0.94}
```

POST /v2/verify_embedding — 1:1 (confirm identity)

Multipart fields: **uid**, **batch**, **image**. Compares the image only against that specific person's enrolled embedding.

Response:

```
{"success": true, "matched": true, "confidence": 0.9954, "threshold": 0.94, "name": "...",
"uid": "..."}
```

Response (uid not found in batch):

```
{"success": false, "error": "No user found: <uid>"} — HTTP 404
```

4. Slap endpoints

Same request shape as the existing minutiae slap endpoints: multiple images in the image field (one per finger, in capture order), plus **hand_side** ("left"/"right", default "right") and an optional **finger_order** (comma-separated finger-position names) if your capture order doesn't match the default 4-finger layout. Each candidate's score is the average of their top 2 per-finger cosine-similarity scores — same aggregation rule as the minutiae slap path.

POST /v2/enroll_slap_embedding

Multipart form fields:

Field	Required	Notes
image	Yes (multiple)	One image per finger, same field name repeated
uid	Yes	Unique person identifier
name	No	Person's name
batch	No	Default "default"
hand_side	No	"left" or "right", default "right"
finger_order	No	Comma-separated finger-position override

Response:

```
{"success": true, "uid": "...", "name": "...", "batch": "...", "hand_side": "right",
"finger_count": 4, "enrolled_fingers": [{"finger_position": "right_little", "iso_code": 5,
"embedding_dim": 512}, ...]}
```

POST /v2/authenticate_slap_embedding — 1:N (identify)

Multipart fields: **batch**, **hand_side** (optional), **image** (multiple). Compares against every person enrolled in the batch.

Response (match found):

```
{"success": true, "uid": "...", "name": "...", "hand_side": "right", "confidence": 0.9957,
"avg_confidence": 0.9957, "threshold": 0.94, "matched_fingers": [{"probe_position":
"right_little", "matched_position": "right_little", "confidence": 0.9954}, ...]}
```

Response (no match):

```
{"success": false, "message": "No match found", "confidence": 0.61, "threshold": 0.94}
```

POST /v2/verify_slap_embedding — 1:1 (confirm identity)

Multipart fields: **uid**, **batch**, **hand_side** (optional), **image** (multiple). Compares only against that specific person's enrolled fingers.

Response:

```
{"success": true, "matched": true, "uid": "...", "name": "...", "hand_side": "right",
"confidence": 0.9957, "avg_confidence": 0.9957, "threshold": 0.94, "matched_fingers":
[...]}
```

5. Reading the response

confidence is a cosine similarity score from -1 to 1 (in practice you'll see roughly 0.85-1.0 for fingerprint photos, genuine or not — this embedding space sits in a narrow, high band, not spread across the full range). **matched** / whether a candidate is returned at all is decided by comparing confidence against **threshold** (currently 0.94, also returned in every response so the app never needs to hardcode it).

6. Important — thresholds are not fully calibrated yet

0.94 was measured on a lab dataset (HKPU contactless fingerprint database — different camera, lighting, and preprocessing than this app's actual capture pipeline) using the **single-finger** protocol. It's a reasonable starting point, not a final production number. The slap threshold currently defaults to that same 0.94 (EMBEDDING_SLAP_THRESHOLD, independently configurable server-side) but has no slap-specific measurement behind it at all yet — it's a placeholder, not even a lab-validated number the way the single-finger one is. Expect both to change once they're measured against real captures from the app — build against the **threshold** field in each response rather than hardcoding 0.94 anywhere in the app, so a server-side recalibration doesn't require a client update.

7. Example (curl)

Enroll (single-finger):

```
curl -X POST https://34-100-150-103.sslip.io/v2/enroll_embedding \
  -F "name=Jane Doe" -F "uid=U001" -F "batch=demo" \
  -F "image=@fingerprint.jpg"
```

Verify (single-finger):

```
curl -X POST https://34-100-150-103.sslip.io/v2/verify_embedding \
  -F "uid=U001" -F "batch=demo" -F "image=@fingerprint2.jpg"
```

Enroll (slap, 4 fingers):

```
curl -X POST https://34-100-150-103.sslip.io/v2/enroll_slap_embedding \
  -F "uid=U001" -F "name=Jane Doe" -F "batch=demo" -F "hand_side=right" \
  -F "image=@little.jpg" -F "image=@ring.jpg" \
  -F "image=@middle.jpg" -F "image=@index.jpg"
```

8. Which one should the app actually call?

Both matchers are live and independent — the minutiae ones and these embedding ones don't share data, so enrolling in one doesn't enroll in the other (same rule for slap: the minutiae slap tables and the embedding slap tables are entirely separate). If you want to compare them side-by-side in the app during evaluation, call both endpoints with the same image(s) and show both results; for a single production flow, use whichever one the team decides on after the thresholds are recalibrated on real data (see §6) — that decision isn't made yet, so don't hardcode a permanent choice. For slap specifically, remember the accuracy itself hasn't been validated yet either, so this isn't ready to be the sole production path regardless of threshold tuning.

Backend: backend/embedding_matcher.py + backend/slap_embedding_matching.py + backend/cloud_app.py (branch siamese-embedding). Validation report: siamese_embedding/Siamese_Embedding_Phase_A_Report.pdf.